

K-means vs DBSCAN

Algorytmy Uczenia Nienadzorowanego — Analiza Techniczna i Porównanie

K-means

vs

DBSCAN

Klastrowanie

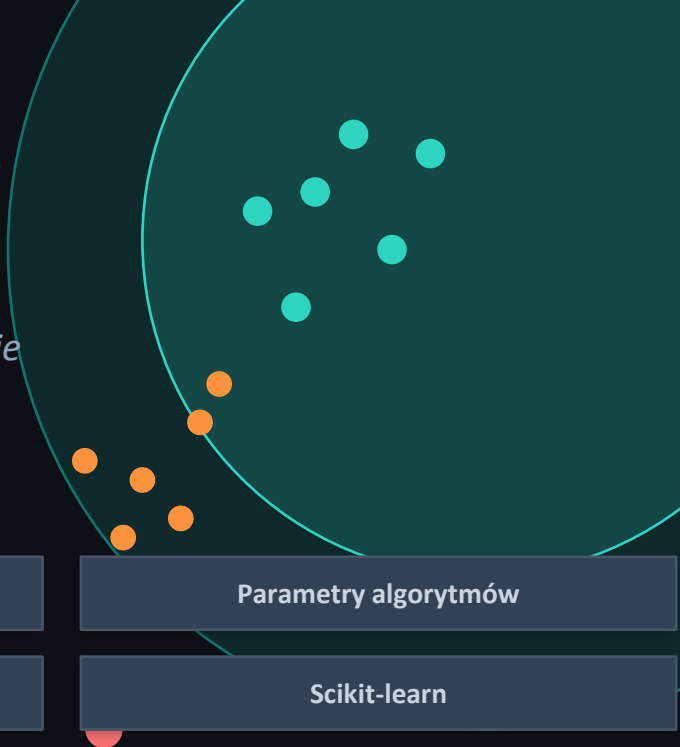
Uczenie nienadzorowane

Parametry algorytmów

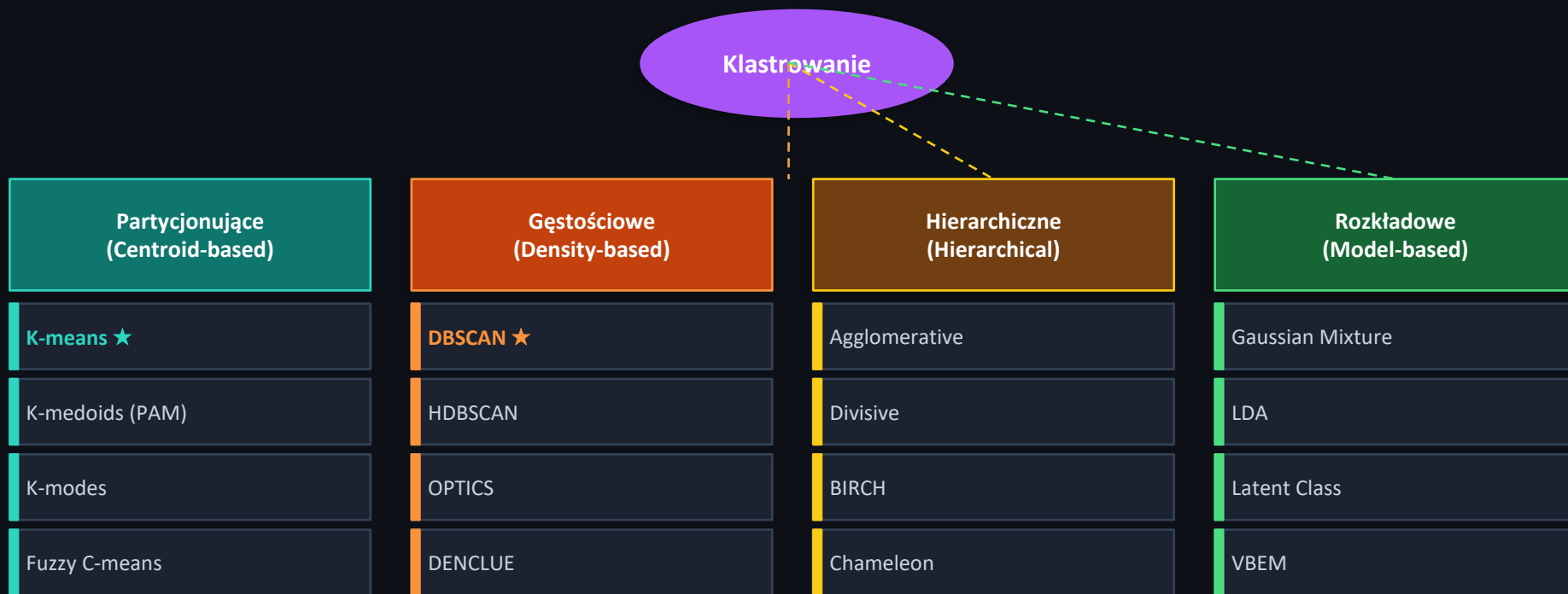
Złożoność obliczeniowa

Obsługa szumu

Scikit-learn



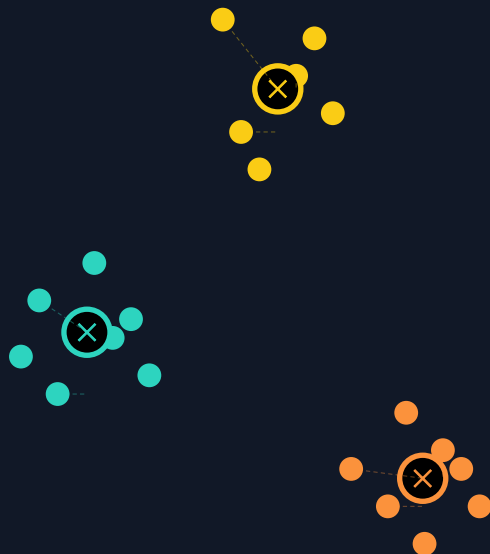
Taksonomia Algorytmów Klastrowania



★ = algorytmy omawiane szczegółowo w tej prezentacji · Uczenie nienadzorowane: brak etykiet — model sam odkrywa strukturę

K-means — Intuicja i Działanie

Przykład: 3 klastry w 2D



× Centroid • Punkt danych

Kluczowa idea K-means

Podziel n punktów na k klastrów tak, aby minimalizować sumę kwadratów odległości od centroidów (WCSS).

Funkcja celu (WCSS):

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

gdzie μ_i = centroid klastra C_i

Typ klastrów:	Wypukłe (convex), sferyczne
Kształt:	Zakłada podobny rozmiar klastrów
Szum / outliers:	Brak obsługi — każdy punkt → klaster
Wejście:	k = liczba klastrów (wymagane!)
Inicjalizacja:	Losowa lub K-means++
Zbieżność:	Gwarantowana (do minimum lokalnego)

K-means — Algorytm Lloyd'a (Pseudokod i Kroki)

1 Inicjalizacja

Wybierz k centroidów $\mu_1 \dots \mu_k$ losowo lub metodą K-means++ (D^2 sampling dla lepszego startu).

2 Krok przypisania (E-step)

Przypisz każdy punkt x_i do najbliższego centroidu:
 $c_j = \operatorname{argmin}_j \|x_i - \mu_j\|^2$ (odległość Euklidesowa)

3 Krok aktualizacji (M-step)

Zaktualizuj centroidy jako średnią punktów klastra:

$$\mu_i = (1/|C_i|) \cdot \sum_{x \in C_i} x$$

4 Test zbieżności

Sprawdź: $\Delta(\mu) < \text{tol}$ lub $\text{iteracja} > \text{max_iter}$.
Jeśli spełnione $\rightarrow$ STOP, inaczej wróć do kroku 2.

Pseudokod — K-means Lloyd

```
function kmeans(X, k, max_iter, tol):
    # Inicjalizacja
     $\mu \leftarrow \text{initialize\_centroids}(X, k)$     # random / k-means++

    for t in 1..max_iter:
        # E-step: przypisanie
        for i in 1..n:
             $C[i] \leftarrow \text{argmin}_j \text{dist}(X[i], \mu[j])$ 

        # M-step: aktualizacja centroidów
         $\mu_{\text{new}} \leftarrow [\text{mean}(X[C==j]) \text{ for } j \text{ in } 1..k]$ 

        # Kryterium zbieżności
        if  $\max(\|\mu_{\text{new}} - \mu\|) < \text{tol}$ :
            break

         $\mu \leftarrow \mu_{\text{new}}$ 

    return C,  $\mu$ 
```

Złożoność: $O(n \cdot k \cdot d \cdot i)$ n=punkty k=klastry
 d>wymiary i=iteracje

K-means — Parametry Algorytmu (scikit-learn)

`n_clusters (k)`

REQUIRED

default: 8

type: `int`

Liczba klastrów. Najważniejszy parametr — musi być podany przez użytkownika.

Elbow Method: wykres WCSS vs $k \rightarrow$ szukaj 'łokcia'.

Silhouette Score: max dla optymalnego k .

Gap Statistic, Davies-Bouldin Index.

`n_init`

optional

default: 10

type: `int`

Liczba niezależnych uruchomień z różnymi centroidami startowymi.

Wyższe `n_init` $\rightarrow$ lepsza jakość, wyższy czas.

Zwraca run z najniższym WCSS.

Dla `k-means++`: `n_init=1` często wystarczy.

`tol`

optional

default: `1e-4`

type: `float`

Względna tolerancja dla kryterium zbieżności (zmiana norm centroidów).

Mniejsze `tol` $\rightarrow$ dokładniejsza zbieżność, wolniej

`init`

REQUIRED

default: `'k-means++'`

type: `str/array`

Metoda inicjalizacji centroidów.

`'k-means++'`: D^2 sampling — centroidy rozłożone daleko od siebie.

`'random'`: losowe punkty z X .

array: własne centroidy startowe.

`max_iter`

optional

default: 300

type: `int`

Maksymalna liczba iteracji dla jednego uruchomienia.

Dla małych danych: 100 wystarczy.

Dla dużych danych: może wymagać więcej.

Jeśli nie osiąga zbieżności $\rightarrow$ ostrzeżenie.

`algorithm`

optional

default: `'lloyd'`

type: `str`

Wariant algorytmu wewnętrznego.

`'lloyd'`: klasyczny EM (domyślny)

```
from sklearn.cluster import KMeans | KMeans(n_clusters=3, init='k-means++', n_init=10, max_iter=300, tol=1e-4)
```

K-means — Warianty i Rozszerzenia

Inicjalizacja

K-means++

Lepsza inicjalizacja centroidów metodą D^2 sampling.

Arthur & Vassilvitskii (2007): $O(\log k)$ -approximation dla optymalnego rozwiązania. 2–3× mniej iteracji.

```
init='k-means++'
```

Skalowalność

Mini-Batch K-means

Aktualizacja centroidów na losowych mini-batchach zamiast całego X .

$O(n) \rightarrow O(\text{batch_size} \cdot t)$. Dla $n > 10^6$ konieczny. Nieznaczna utrata jakości klasteryzacji.

```
MiniBatchKMeans(batch_size=1024)
```

Odporność

K-medoids (PAM)

Centroid = rzeczywisty punkt danych (medoid) — nie średnia.

Odporny na outliers (brak wpływu wartości skrajnych). Dowolna metryka odległości. Wolniejszy: $O(n^2)$.

```
sklearn_extra.cluster.KMedoids
```

Przynależność rozmyta

Fuzzy C-means

Punkt należy do wielu klastrów z różnym stopniem przynależności.

Gdy punkty leżą blisko granicy klastrów. Brak twardego podziału — probabilistyczna przynależność.

```
skfuzzy.cluster.cmeans (scikit-fuzzy)
```

Hierarchiczny

Bisecting K-means

Rekurencyjny podział: zaczynaj od 1 klastra, dziel największy.

Lepsza od k-means dla niespójnych klastrów. Mniejsza wrażliwość na inicjalizację.

```
BisectingKMeans(n_clusters=k)
```

Nieliniowy

Kernel K-means

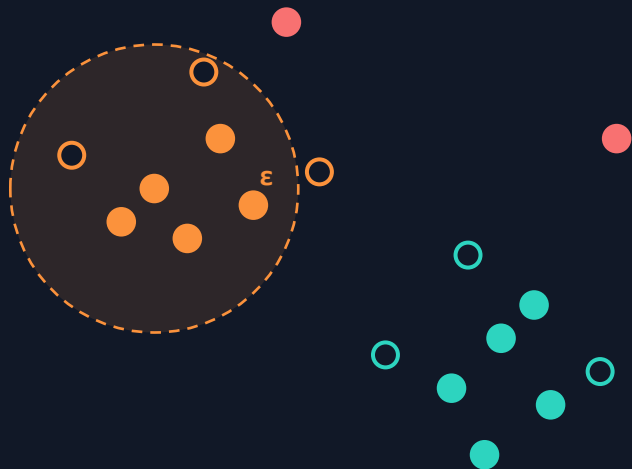
K-means w przestrzeni cech wyindukowanej przez jądro.

Dla klastrów o kształtach niewypukłych (księżyc, spirala). Ekwiwalent spektral clustering z RBF kernel.

Implementacje: SpectralClustering

DBSCAN — Intuicja i Kluczowe Pojęcia

Typy punktów w DBSCAN



•=Core o=Border •=Noise

Core Point (punkt rdzeniowy)

Punkt p jest core pointem $\Leftrightarrow$

$$|N_{\epsilon}(p)| \geq \text{MinPts}$$

$$\text{gdzie } N_{\epsilon}(p) = \{q \in X : \text{dist}(p, q) \leq \epsilon\}$$

Ma wystarczająco wielu sąsiadów w promieniu ϵ .

Border Point (punkt brzegowy)

Punkt q jest border pointem $\Leftrightarrow$

- q NIE jest core pointem ($|N_{\epsilon}(q)| < \text{MinPts}$)
- q jest osiągalny z jakiegoś core pointa p

Leży na granicy klastra.

Noise Point (szum)

Punkt r jest szumem $\Leftrightarrow$

- NIE jest core pointem
- NIE jest osiągalny z żadnego core pointa

Oznaczony jako -1. Outlier.

DBSCAN — Algorytm (Definicje i Pseudokod)

Formalne Definicje

ϵ -sąsiedztwo $N_\epsilon(p)$

$\{ q \in D : \text{dist}(p,q) \leq \epsilon \}$

Zbiór punktów w promieniu ϵ od p

Bezpośrednia osiągalność

$\text{dist}(p,q) \leq \epsilon \wedge |N_\epsilon(p)| \geq \text{MinPts}$

q jest bezpośred. osiągalny z p

Osiągalność gęstościowa

$p \rightarrow_d q$: istnieje łańcuch $p=p_1 \dots p_n=q$

q jest osiągalny z p przez chain

Połączenie gęstościowe

$\exists o: p \leftrightarrow_d o \rightarrow_d q$

p i q połączone przez wspólny rdzeń

Klaster C

$\forall p, q \in C: p \leftrightarrow q$ (density-connected)

Maksymalny zbiór density-connected

Pseudokod — DBSCAN (Ester et al., 1996)

```
function dbscan(X,  $\epsilon$ , MinPts):
    labels = [-1] * len(X)          # -1 = szum
    cluster_id = 0

    for p in X:
        if labels[p] != -1: continue # odwiedzone
        N = range_query(X, p,  $\epsilon$ )
        if len(N) < MinPts:
            labels[p] = NOISE; continue
        cluster_id += 1
        labels[p] = cluster_id
        seeds = N - {p}

        while seeds !=  $\emptyset$ :
            q = seeds.pop()
            if labels[q] == NOISE:
                labels[q] = cluster_id
            if labels[q] != -1: continue
            labels[q] = cluster_id
            N_q = range_query(X, q,  $\epsilon$ )
            if len(N_q) >= MinPts:
                seeds = seeds + N_q

    return labels
```

Złożoność: $O(n \cdot \log n)$ z indeksem przestrzennym (R-tree, k-d tree) $\cdot O(n^2)$ naiwnie

DBSCAN — Parametry Algorytmu (szczegółowo)

Parametr: ϵ (eps) — promień sąsiedztwa

Definicja:	Maksymalna odległość między dwoma punktami, aby były uznane za sąsiadów. Wartość realna > 0 .
Wpływ ϵ za małe:	Prawie wszystkie punkty = szum (-1). Klastry rozpadają się na wyspy. Ekstremalnie: n klastrow 1-elementowych.
Wpływ ϵ za duże:	Wszystkie punkty w jednym klastrze. Nie wykrywa gęstości lokalnej. Ekstremalnie: 1 klaster.
Dobór ϵ:	k-distance plot: posortuj odległości k-tego sąsiada. Szukaj 'łokcia' wykresu. $k = \text{MinPts}$.
Wskazówka:	Znormalizuj dane przed DBSCAN. ϵ zależy od skali. Użyj StandardScaler lub MinMaxScaler.

Parametr: MinPts (min_samples) — minimalna liczba punktów

Definicja:	Minimalna liczba punktów (włącznie z p) w ϵ -sąsiedztwie, żeby p był core pointem. Liczba całkowita ≥ 1 .
Wpływ MinPts=1:	Każdy punkt jest core pointem. Brak szumu. Ekstremalnie wrażliwy klaster.
Wpływ MinPts za duże:	Mało core pointów. Wiele punktów = szum. Małe prawdziwe klastry znikają.
Reguła praktyczna:	$\text{MinPts} \geq d + 1$, gdzie d = wymiarowość danych (Sander et al.). Dla 2D: $\text{MinPts}=4$. Dla 10D: $\text{MinPts} \geq 11$.
<code>metric (default='euclidean'):</code> 'manhattan', 'cosine', 'haversine' (GPS!), 'precomputed', callable — dowolna funkcja dystansu	

DBSCAN — Warianty i Rozszerzenia

HDBSCAN

Hierarchical DBSCAN

Rozszerzenie DBSCAN, które nie wymaga parametru ϵ . Buduje hierarchię klastrow gęstościowych i automatycznie wybiera stabilne klastry.

Parametry:

$\min_cluster_size$: min. rozmiar klastra
 $\min_samples$: analogiczne do MinPts
 α : konserwatywność

```
from hdbscan import HDBSCAN
HDBSCAN(min_cluster_size=5)
```

OPTICS

Ordering Points To Identify Clustering Structure

Generalizacja DBSCAN. Tworzy wykres osiągalności (reachability plot) kodujący strukturę klastrow dla wielu wartości ϵ naraz.

Parametry:

$\min_samples$: jak MinPts w DBSCAN
 $\max_eps$: ograniczenie promienia
 ξ : stromość wykresu dla klastrow
 $cluster_method$: 'xi' lub

```
from sklearn.cluster import OPTICS
OPTICS(min_samples=5, xi=0.05)
```

DENCLUE

DENSITY-based CLUstEring

Modeluje funkcję gęstości jako sumę funkcji jądrowych (kernel density estimation). Klastry to baseny atrakcji gradientowego wspinania.

Parametry:

h : szerokość jądra (bandwidth)
 ξ : próg gęstości
Analogiczne do ϵ i MinPts

Brak sklearn. Pakiet: `pyclustering`

ST-DBSCAN

Spatio-Temporal DBSCAN

Rozszerzenie DBSCAN do danych przestrzenno-czasowych. Używa dwóch parametrów ϵ : jeden dla przestrzeni, drugi dla czasu.

Parametry:

ϵ_1 : promień przestrzenny (np. km GPS)
 ϵ_2 : próg czasowy (np. godziny/dni)
 $\min_pts$: min. sąsiadów w obu

Implementacja własna lub `bigspatialdata/ST-DBSCAN`

DBSCAN++

Approximate DBSCAN

Aproksymacja DBSCAN dla dużych danych. Losowo próbkuje rdzeń punktów i rozszerza klastry tylko z próbki.

Parametry:

ϵ , MinPts: jak w DBSCAN
 m : rozmiar próbki (fraction of n)
Kompromis: szybkość vs jakość

Brak oficjalny. Implementacje [GitHub](#).

LDBSCAN

Local Density-based DBSCAN

Adaptacyjne ϵ : każdy punkt ma własny lokalny promień oparty o k-NN. Radzi sobie z klastrami o bardzo różnych gęstościach.

Parametry:

k : liczba najbliższych sąsiadów
 $\min_pts$: jak w DBSCAN
 $local_scale$: metoda adaptacji ϵ

Implementacje badawcze / własna.

K-means vs DBSCAN — Porównanie Techniczne

Kryterium	K-means	DBSCAN
Parametry wejściowe	k (liczba klastrów) — wymagane	ϵ , MinPts — wymagane
Automatyczny dobór k	NIE — k zawsze podane przez użytk.	TAK — liczba klastrów emergentna
Kształt klastrów	Tylko wypukłe (convex), sferyczne	Dowolny kształt (concave, ring...)
Obsługa szumu / outlierów	BRAK — każdy punkt → jakiś klaster	TAK — punkty szumu oznaczone -1
Zmienna gęstość klastrów	Zakłada podobny rozmiar	PROBLEM — trudne dla różnych ϵ
Złożoność czasowa	$O(n \cdot k \cdot d \cdot i)$ — szybki dla dużych n	$O(n^2)$ / $O(n \log n)$ z indeksem
Skalowalność ($n > 10^6$)	TAK (MiniBatch K-means)	TRUDNE — indeks przestrzenny konieczny
Wysoka wymiarowość	Umiarkowanie OK (curse of dim.)	PROBLEM — sparseness, dobór ϵ
Deterministyczność	NIE — wynik zależy od init	TAK (dla ustalonego ϵ , MinPts)
Interpretowalność	Wysoka — centroid = centrum	Średnia — brak centrum klastra
Metryka odległości	Domyślnie Euklidesowa (twarda)	Dowolna (Cosine, Manhattan...)
Soft / fuzzy clustering	TAK (Fuzzy C-means wariant)	NIE (twarde przypisanie)

Kiedy Użyć K-means, a Kiedy DBSCAN?



Implementacja Python — scikit-learn (Kompletny Przykład)

K-means — Pełny Pipeline

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

# 1. Generuj dane
X, _ = make_blobs(n_samples=500, centers=4,
                  cluster_std=0.8, random_state=42)

# 2. Normalizacja (ważne!)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Elbow Method — dobór k
wcss = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, init='k-means++',
                n_init=10, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
```

```
# 4. Dopasowanie modelu
```

DBSCAN — Pełny Pipeline + k-distance

```
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
import matplotlib.pyplot as plt

# 1. Dobór  $\epsilon$  metodą k-distance plot
k = 4 # MinPts
nbrs = NearestNeighbors(n_neighbors=k)
nbrs.fit(X_scaled)
distances, _ = nbrs.kneighbors(X_scaled)
k_distances = np.sort(distances[:, k-1])[:, :-1]

# Wykres — szukaj 'łokcia'
plt.plot(k_distances)
plt.xlabel('Punkty (posortowane)')
plt.ylabel(f'{k}-NN odległość')
plt.title('k-distance plot — dobór  $\epsilon$ ')

# 2. Dopasowanie DBSCAN
db = DBSCAN(eps=0.3, # dobrane z wykresu
            min_samples=4, # MinPts = k
            metric='euclidean',
            algorithm='auto') # auto→k-d tree

labels = db.fit_predict(X_scaled)
```

Ewaluacja Klasteryzacji — Metryki i Indeksy

Metryki wewnętrzne (bez etykiet referencyjnych) — używane gdy brak ground truth

Silhouette Score

[-1, 1]

→ 1

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

$a(i)$: śr. odległość do punktów w tym samym klastrze

$b(i)$: śr. odległość do punktów w najbliższym klastrze

Interpretacja: >0.5 dobry, >0.7 świetny, <0 zły

→ Działa dla K-means i DBSCAN (ignoruj szum -1)

Davies-Bouldin Index

[0, ∞)

→ 0

$$DB = (1/k) \sum_i \max_{j \neq i} (\sigma_i + \sigma_j) / d(\mu_i, \mu_j)$$

σ_i : śr. odległość punktów klastra i od centroidu

$d(\mu_i, \mu_j)$: odległość między centroidami

Interpretacja: mniejszy = lepszy

→ Lepszy dla K-means. Wymaga centroidów.

Calinski-Harabasz (CH)

[0, ∞)

→ max

$$CH = [SSB/(k-1)] / [SSW/(n-k)]$$

SSB: rozproszenie między klastrami (between)

SSW: rozproszenie wewnątrz klastrów (within)

Im wyższy — tym lepsze oddzielenie

→ Szybki, ale faworyzuje klastry wypukłe (K-means bias)

Inertia / WCSS

[0, ∞)

→ 0

$$WCSS = \sum_i \sum_{x \in C_i} \|x - \mu_i\|^2$$

Suma kwadratów odległości od centroidów.

Używana w Elbow Method dla K-means.

Nie porównywalny między różnymi k !

→ TYLKO K-means. Maleje monotonicznie z k .

Metryki zewnętrzne (gdy dostępne etykiety ground truth): Adjusted Rand Index (ARI) · Normalized Mutual Information (NMI) · Homogeneity · V-measure

```
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score, homogeneity_completeness_v_measure
silhouette_score(X, labels, metric='euclidean') # Silhouette
```

Podsumowanie — Kluczowe Wnioski

K-means — Praktyczne Wskazówki

- ✓ Wymaga podania k z góry — dobierz Elbow Method lub Silhouette Score
- ✓ Zakłada klastry sferyczne i podobnego rozmiaru — sprawdź dane
- ✓ Deterministyczny wynik TYLKO z fixed random_state
- ✓ Używaj init='k-means++' i n_init≥10 zawsze
- ✓ Normalizacja StandardScaler przed fit() — obowiązkowa
- ✓ MiniBatchKMeans dla $n > 100\,000$ punktów

DBSCAN — Praktyczne Wskazówki

- ✓ Dobierz ϵ z k-distance plot (łokieć = ϵ), MinPts $\geq d+1$
- ✓ Wynik etykiety: 0,1,2,... = klastry, -1 = outliers/szum
- ✓ Zeskaluj dane — ϵ jest absolutny, zależy od skali
- ✓ HDBSCAN dla zmiennych gęstości, OPTICS dla eksploracji
- ✓ algorithm='ball_tree' lub 'kd_tree' dla dużych n
- ✓ Silhouette ignoruj punkty szumu: score(X[labels!= -1], ...)